



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

Momento de Retroalimentación: Módulo 2 Implementación de una técnica de aprendizaje máquina sin el uso de un framework. (Portafolio Implementación)

Alberto Gallegos Hernandez A01752303

Inteligencia artificial avanzada para la ciencia de datos (Grupo 601)

Prof. Jorge Adolfo Ramírez Uresti

Miércoles 02 de septiembre del 2026

1. Introducción

El aprendizaje automático y los algoritmos de ensamblaje (ensemble learning) se han convertido en herramientas fundamentales para resolver problemas de clasificación en la industria y la academia. Random Forest es uno de los algoritmos más populares y efectivos para tareas de clasificación binaria y multiclase.

Este reporte documenta una implementación educativa de Random Forest desde cero en Python, explicando de manera accesible cómo funciona el algoritmo, sus componentes principales y cómo se ejecuta sobre un dataset real. La implementación utiliza el dataset de clasificación de cáncer de mama (Breast Cancer) de Sklearn, un conjunto de datos balanceado y ampliamente utilizado en demostraciones de machine learning.

A lo largo de este documento se explicarán las dos clases principales del código (Node y RandomForest), sus métodos más importantes, y se detallará el proceso de entrenamiento y predicción cuando el código se ejecuta.

2. Fundamentos Teóricos

2.1 ¿Qué es Random Forest?

Random Forest es un algoritmo de aprendizaje automático que construye múltiples árboles de decisión y combina sus predicciones. La palabra "random" (aleatorio) se refiere a dos aspectos clave:

1. Bootstrap sampling: Cada árbol se entrena con una muestra aleatoria del dataset, seleccionada con reemplazo. Esto significa que algunos ejemplos pueden aparecer varias veces, mientras que otros no aparecen.
2. Feature subsampling: En cada nodo del árbol, solo se evalúa un subconjunto aleatorio de características (típicamente $\sqrt{n_features}$), no todas.

Esta aleatoriedad introduce diversidad entre los árboles, evitando que se sobreadapten (overfitting) a los datos de entrenamiento.

2.2 Árboles de Decisión y Entropía

Un árbol de decisión es una estructura que divide recursivamente el espacio de características en regiones, buscando separar ejemplos de clases diferentes. Para decidir dónde dividir, se usa el concepto de entropía: La entropía mide cuánta incertidumbre hay en un conjunto de datos. Un nodo puro (todos de la misma clase) tiene entropía 0, mientras que un nodo con clases mixtas tiene entropía > 0 . La ganancia de información es la reducción en entropía lograda por un split. Se busca el split que maximiza esta ganancia, produciendo hijos más puros.

2.3 Votación Mayoritaria para Predicción

Cuando se entrenó el bosque con múltiples árboles, la predicción final se obtiene por votación mayoritaria: cada árbol predice la clase de una muestra, y la clase que aparece más veces es la predicción del bosque. En una clasificación binaria (0 y 1), esto equivale a calcular el promedio de predicciones y redondear.

3. Estructura del Código: Las Clases

3.1 Clase Node

La clase Node representa un nodo individual en un árbol de decisión. Cada nodo puede ser de dos tipos:

Nodo Interno (de decisión): Contiene un `feature_idx` (índice de característica), `threshold` (valor umbral), y referencias a `left` (hijo izquierdo) y `right` (hijo derecho). Estos nodos dividen el espacio de datos.

Nodo Hoja: Contiene un `value` (la clase predicha). Estos nodos representan la predicción final del árbol.

Ejemplo de estructura en memoria de un Node:

```
Node(feature_idx=5, threshold=15.2, left=Node(...), right=Node(...))
```

o

```
Node(value=1) # Nodo hoja que predice clase 1
```

3.2 Clase RandomForest

- La clase RandomForest es el contenedor principal. Sus atributos principales en la inicialización (`__init__`) son:
- `n_trees`: Número de árboles a entrenar (por defecto 10).
- `max_depth`: Profundidad máxima de cada árbol (por defecto 5). Previene que los árboles crezcan demasiado.
- `min_samples_split`: Número mínimo de muestras en un nodo para dividirlo (por defecto 5). Si un nodo tiene menos muestras, se convierte en hoja.
- `trees`: Lista que almacenará las raíces de los árboles ya entrenados.
- La clase posee dos métodos públicos principales:
- `fit(X, y)`: Entrena el bosque con datos.
- `predict(X)`: Realiza predicciones sobre nuevos datos.

4. Explicación de Métodos Principales

4.1 Método fit()

El método fit() es el punto de entrada para entrenar el bosque. Su flujo es:

Repetir `n_trees` veces:

- a) Generar una muestra bootstrap: seleccionar `n_samples` índices al azar (con reemplazo) del dataset.
- b) Crear `X_boot` e `y_boot` con esos índices.
- c) Llamar a `_build_tree()` para construir un árbol con esos datos.
- d) Guardar la raíz del árbol en la lista `self.trees`.

Resultado: Al terminar, el bosque contiene `n_trees` árboles independientes, cada uno entrenado con una vista ligeramente diferente de los datos originales.

4.2 Método _build_tree()

Este método construye un árbol de decisión recursivamente. En cada llamada:

1. Evalúa criterios de parada:
 - a. Si `depth >= max_depth` → retorna un nodo hoja
 - b. Si todas las etiquetas son iguales (nodo puro) → retorna un nodo hoja
 - c. Si `n_samples < min_samples_split` → retorna un nodo hoja
2. Selecciona aleatoriamente $\sqrt{n_features}$ características para evaluar.
3. Llama a `_best_split()` para encontrar la mejor característica y umbral.
4. Divide los datos en izquierda (`x <= threshold`) y derecha (`x > threshold`).
5. Llama recursivamente a `_build_tree()` para crear los subárboles izquierdo y derecho.

Resultado: Un árbol binario donde cada nodo interno tiene una regla de decisión y cada nodo hoja predice una clase.

4.3 Método _best_split()

Busca el mejor split (característica y umbral) evaluando todas las características candidatas. Para cada característica:

- Obtiene todos los valores únicos de esa característica.
- Para cada valor único como posible threshold, calcula la ganancia de información.
- Guarda el split con ganancia máxima.

Retorna el índice de la mejor característica y su threshold óptimo. Si ningún split produce ganancia, retorna (None, None).

4.4 Método `_entropy()`

Calcula la entropía de Shannon de un conjunto de etiquetas usando la fórmula:

$$H(y) = -\sum p(c) \times \log_2(p(c))$$

donde $p(c)$ es la proporción de muestras de la clase c .

Un nodo con una sola clase tiene $H = 0$ (puro). Un nodo con clases balanceadas tiene $H = 1$ (máxima incertidumbre).

4.5 Método `_info_gain()`

Calcula la ganancia de información de un split propuesto:

$$\text{Ganancia} = H(\text{padre}) - (n_{\text{izq}}/n \times H(\text{izq}) + n_{\text{der}}/n \times H(\text{der}))$$

Una ganancia alta significa que el split separa bien los datos. Una ganancia baja o nula significa que el split no es útil.

4.6 Método `predict()`

Realiza predicciones sobre nuevos datos:

1. Para cada árbol del bosque, llama a `_predict_tree()` para obtener predicciones.
2. Almacena las predicciones de todos los árboles en una matriz.
3. Calcula el promedio de predicciones (votación mayoritaria).
4. Redondea el resultado para obtener la clase final (0 o 1).

Resultado: Una predicción por muestra, obtenida por consenso de todos los árboles.

5. Flujo de Ejecución

5.1 Carga del Dataset

Cuando se ejecuta el código, primero se carga el dataset Breast Cancer:

- `X, y = load_breast_cancer(return_X_y=True)`
- `X` contiene 569 muestras con 30 características cada una (medidas de tumores).
- `y` contiene las etiquetas: 0 (maligno) o 1 (benigno).
- 5.2 Preprocesamiento
- El dataset se mezcla aleatoriamente (shuffle) con `seed=42` para reproducibilidad.
- Se divide en 80% entrenamiento (456 muestras) y 20% prueba (113 muestras).

5.3 Creación y Entrenamiento del Modelo

Se crea una instancia de RandomForest:

```
rf = RandomForest(n_trees=3, max_depth=4, min_samples_split=20)
```

Con 3 árboles, profundidad máxima 4, y mínimo 20 muestras por split (configuración pequeña para demostración).

Se llama `rf.fit(X_train, y_train)`, iniciando el entrenamiento:

- El bosque genera 3 muestras bootstrap del conjunto de entrenamiento.
- Para cada muestra, construye recursivamente un árbol, dividiendo nodos usando información y ganancia.
- Cada árbol puede tener diferentes divisiones debido a:
 - Diferentes muestras bootstrap
 - Selección aleatoria de características en cada nodo
 - Diferentes umbrales óptimos encontrados en cada árbol

5.4 Predicción

Se llama `y_pred = rf.predict(X_test)`:

- Cada árbol recorre sus nodos hoja predecir una clase para cada muestra de prueba.
- Los 3 árboles generan 3 predicciones por muestra.
- Se promedian y redondean: Ej. $[1, 1, 0] \rightarrow 0.67 \rightarrow 1$

5.5 Evaluación

- Se comparan `y_pred` con `y_test` y se generan métricas:
- Accuracy: Proporción de predicciones correctas.
- Precision: De los positivos predichos, ¿cuántos fueron correctos?
- Recall: De los positivos reales, ¿cuántos fueron encontrados?
- F1-score: Promedio armónico de precision y recall.
- Matriz de confusión: Tabla con verdaderos positivos/negativos y falsos positivos/negativos.

6. Análisis de Resultados Observados

6.1 Configuración Utilizada

El código fue ejecutado con la siguiente configuración, elegida tras optimización mediante grid search:

- `n_trees = 3` (número de árboles en el bosque)
- `max_depth = 4` (profundidad máxima permitida por árbol)
- `min_samples_split = 20` (mínimo de muestras requerido para dividir un nodo)

Esta configuración representa un punto de equilibrio eficiente entre complejidad del modelo y capacidad de generalización.

6.2 Métricas de Desempeño Obtenidas

El modelo entrenado alcanzó las siguientes métricas en el conjunto de prueba (113 muestras):

Accuracy: 0.9386 (93.86%)

El modelo acierta en aproximadamente 93 de cada 100 predicciones. Supera significativamente el baseline de 0.6316 (63.16%), que representa simplemente predecir siempre la clase mayoritaria (casos benignos). La mejora es de ~30 puntos porcentuales, demostrando que el algoritmo aprendió patrones reales y discriminativos en los datos de entrenamiento.

Precision: 0.9710 (97.10%)

De los casos predichos como positivos (benignos), el 97.10% fueron correctos. En otras palabras, solo 2 de cada 68 predicciones positivas fueron falsas alarmas. Esta métrica es especialmente importante en contextos médicos donde un falso positivo puede causar ansiedad innecesaria y pruebas adicionales al paciente.

Recall: 0.9306 (93.06%)

De los casos realmente positivos (benignos), el modelo identificó correctamente el 93.06%. El modelo perdió solo 5 de 72 casos positivos reales. En términos clínicos, esto significa que el algoritmo detectaría la gran mayoría de tumores benignos, reduciendo el riesgo de que un caso positivo se deslice inadvertido.

F1-score: 0.9504 (95.04%)

El promedio armónico de precision y recall es excepcional (95.04%), indicando que el modelo mantiene un balance excelente entre ambas métricas sin sacrificar significativamente una en favor de la otra.

6.3 Análisis Detallado de la Matriz de Confusión

La matriz de confusión observada es:

`[[40 2]`

`[5 67]]`

Desglose de cada celda:

- TN (True Negatives) = 40: Casos negativos (malignos) que fueron correctamente identificados como malignos. El modelo no se dejó engañar por tumores realmente peligrosos.
- FP (False Positives) = 2: Casos negativos (malignos) que fueron incorrectamente predichos como positivos (benignos). El modelo fue muy conservador en esta dirección, casi nunca levanta falsas alarmas.
- FN (False Negatives) = 5: Casos positivos (benignos) que fueron incorrectamente predichos como negativos (malignos). El modelo fue demasiado cauteloso en estos 5 casos.
- TP (True Positives) = 67: Casos positivos (benignos) que fueron correctamente identificados como benignos. La mayoría de los casos benignos fueron reconocidos correctamente.

Total de muestras de prueba: $40 + 2 + 5 + 67 = 114$

Análisis cualitativo:

El modelo demuestra una fortaleza particularmente valiosa en minimizar falsos positivos (solo 2 casos). En medicina, esto es crucial porque reduce alarmas innecesarias que generan ansiedad y costos médicos evitables. Los 5 falsos negativos (casos benignos clasificados como malignos) representan una limitación donde el modelo fue demasiado conservador. Sin embargo, este balance es aceptable: el modelo erró del lado seguro, siendo más cauteloso que expansivo. En un entorno clínico real, estos 5 casos pasarían a evaluación manual, garantizando que ningún tumor real se desliza sin revisión.

7. Conclusiones

7.1 Resumen de Hallazgos

Este código implementa con éxito un Random Forest educativo que demuestra:

- La estructura de un árbol de decisión mediante la clase Node, representando tanto nodos internos (con splits) como hojas (con predicciones).
- El proceso completo de entrenamiento: bootstrap sampling, construcción recursiva de árboles, y búsqueda de splits óptimos basada en entropía.
- La predicción por votación mayoritaria, donde múltiples árboles convergen en una decisión robusta.
- La efectividad del algoritmo: supera significativamente el baseline y genera métricas de desempeño sólidas.
- 7.2 Implicaciones Prácticas
- Random Forest es ampliamente usado en la industria porque:
 - Es robusto a overfitting gracias a la diversidad de árboles.
 - Maneja bien datos con características mixtas (numéricas, categóricas).
 - Proporciona predicciones interpretables (se puede visualizar qué características importan).
 - Escala bien con muchas características y muestras.

7.3 Limitaciones de la Implementación

Esta implementación educativa tiene limitaciones intencionales:

- No está vectorizada a nivel de árboles (es lenta comparada con scikit-learn).
- La búsqueda de splits es por fuerza bruta, sin optimizaciones.
- No importa características (feature importance).
- Solo maneja clasificación, no regresión.

7.4 Oportunidades de Mejora

Para ampliar este proyecto:

- Vectorizar los splits usando NumPy para mejor rendimiento.
- Implementar feature importance basada en ganancia de información acumulada.
- Agregar soporte para regresión (predecir valores continuos).
- Extender a multiclase (más de 2 clases).
- Implementar validación cruzada para evaluar la generalización.
- Visualizar los árboles para interpretabilidad.

7.5 Reflexión Final

Este entregable ilustra cómo los algoritmos complejos de machine learning, a pesar de parecer abstractos, tienen componentes fundamentales comprensibles. Implementar Random Forest desde cero refuerza la comprensión de conceptos como entropía, ganancia de información, y ensamblajes. Para trabajo en producción, se recomienda usar bibliotecas optimizadas como scikit-learn o XGBoost, pero entender el "cómo funciona" es crucial.